Job Market Intelligence Pipeline

System Design Document

High-Level Architecture · Pipeline Run Analysis · Component Reference

Platform	Last run	Status
Databricks (Serverless)	Mar 29, 2026 · 15m 36s	✓ Succeeded

1. Executive Summary

The Job Market Intelligence Pipeline is a fully automated, end-to-end Databricks-native system that scrapes data engineering job postings from Google and Amazon Careers, transforms and stores them as Delta tables, embeds them into a Databricks Vector Search index, and exposes a LangGraph multi-agent interface for natural language job market analysis.

The pipeline ran successfully on March 29, 2026, completing in 15 minutes and 36 seconds across five sequential Databricks job stages on a serverless cluster. The system produced analysis of the current data engineering hiring landscape, culminating in a structured market report and a LinkedIn post.

Design Goals

- Idempotency — all stages use watermark-guarded MERGE operations; safe to re-run at any time.
- Incremental processing — each stage only processes data newer than its watermark lower bound.
- Separation of concerns — ingestion, cleaning, chunking, embedding, retrieval, and reasoning are fully decoupled.
- Observability — every stage writes status, row counts, and error notes to a watermark audit history table.
- Extensibility — adding a new organisation (e.g. Microsoft) requires only a new scraper and watermark registration.

2. Actual Pipeline Run — Databricks Jobs

The following section documents the actual Databricks job run captured in the pipeline screenshot. All five stages completed successfully with a total wall-clock time of 15 minutes 36 seconds.

2.1 Job Run Screenshot

The screenshot below is the Databricks Jobs graph view for the production run of the pipeline. It shows the two parallel ingestion stages (scrape_amazon and scrape_google) converging into a sequential chain of clean_data → chunk → embed.

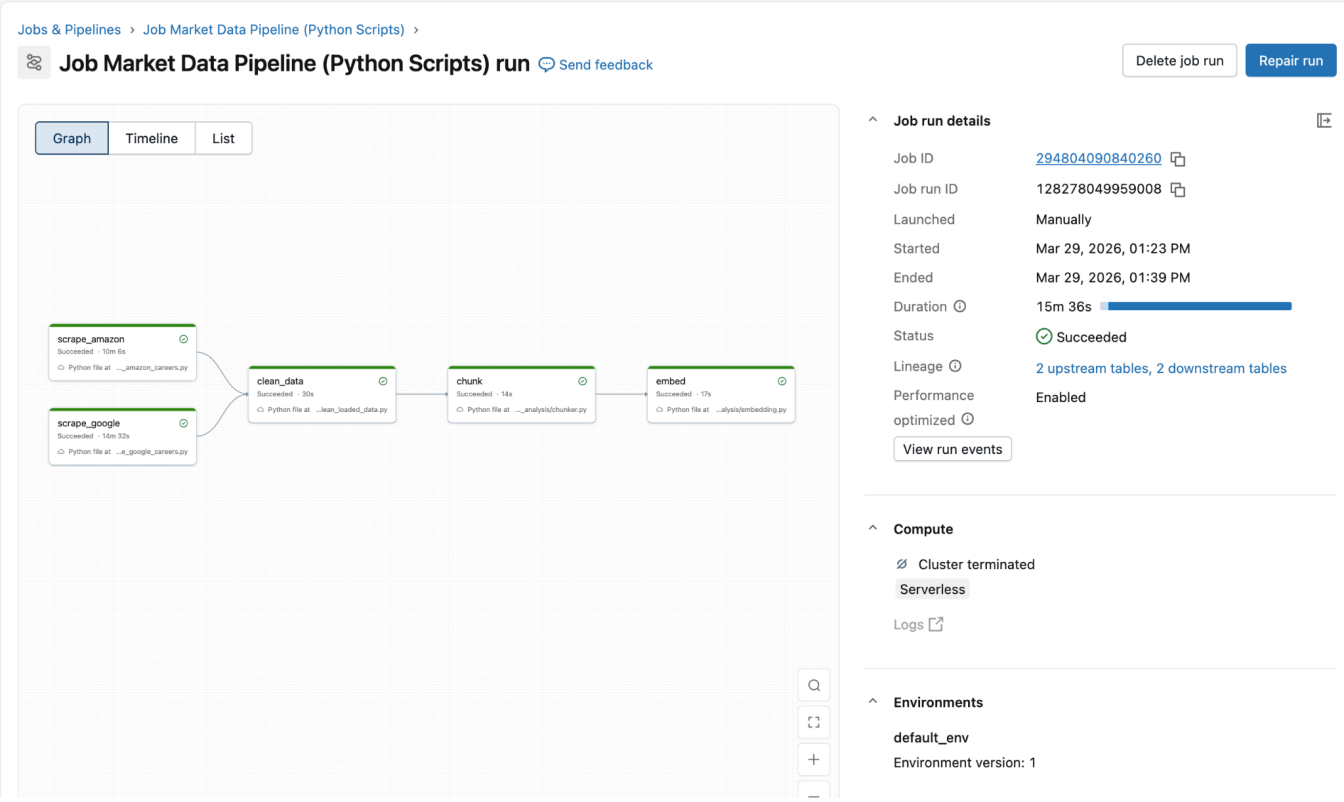

Figure 1 — Databricks Jobs graph view: Job Market Data Pipeline (Python Scripts) run · Mar 29, 2026 · Status: Succeeded · Duration: 15m 36s

2.2 Job Run Details

Job run attribute	Value
Pipeline name	Job Market Data Pipeline (Python Scripts)
Job ID	294804090840260
Job run ID	128278049959008
Launched	Manually
Started	Mar 29, 2026, 01:23 PM
Ended	Mar 29, 2026, 01:39 PM

Duration	15 minutes 36 seconds
Status	Succeeded
Lineage	2 upstream tables, 2 downstream tables
Compute	Serverless (cluster terminated post-run)
Environment	default_env (version 1)
Performance	Optimised — enabled

2.3 Stage Execution Summary

The five stages executed in a fan-in / sequential pattern. Both scrapers ran in parallel (scrape_amazon: 10m 6s; scrape_google: 14m 32s), converging into clean_data once both had merged into raw_openings. Downstream stages were gated by the watermark system.

Stage	Script	Status	Duration	Notes
scrape_amazon	`scrape_amazon_careers.py`	Succeeded	10m 6s	Parallel with scrape_google; MERGE into raw_openings
scrape_google	`scrape_google_careers.py`	Succeeded	14m 32s	Parallel with scrape_amazon; MERGE into raw_openings
clean_data	`clean_loaded_data.py`	Succeeded	30s	Normalise titles, locations, seniority; quarantine bad records
chunk	`chunker.py`	Succeeded	14s	Sentence-window chunking (window=2); MERGE into job_sentence_chunks
embed	`embedding.py`	Succeeded	17s	Trigger DELTA_SYNC VS index; poll until ONLINE_NO_PENDING_UPDATE

3. High-Level Architecture

The pipeline is structured as six logical layers, each reading from and writing to Delta tables as the shared state store. The watermark system gates each layer on its predecessor, ensuring that no stage processes data until the upstream stage has fully committed its output.

3.1 Architecture Layers

Layer	Script(s)	Input → Output	Key design decisions
1 — Ingestion	`scrape_google_careers.py` `scrape_amazon_careers.py`	Career pages → raw_openings (Delta)	Parallel scraping; MERGE on (job_id, org); watermark guard; exponential back-off on HTTP 429
2 — Cleaning	`clean_loaded_data.py`	raw_openings → clean_openings + quarantine_openings	Per-org watermark gate; seniority / domain / job family classification; quarantine for quality failures
3 — Chunking	`chunker.py`	clean_openings → job_sentence_chunks (Delta)	Sentence-window (±2); org-aware section detection; boilerplate strip; MERGE on (doc_id, chunk_id)
4 — Embedding	`embedding.py` + `vs_index.py` + `vs_sync.py`	job_sentence_chunks → job_chunks_index (VS)	DELTA_SYNC TRIGGERED index; databricks-gte-large-en; polls detailed_state; watermark only advances on success
5 — Retrieval	`retriever.py`	User query + VS index → formatted context	Hybrid ANN+BM25; VS pre-filter (org, country, seniority, is_intern); MMR re-rank (λ=0.5)
6 — Multi-agent	`build_agents.py` + `08_run_queries.py`	Formatted context → final_answer + linkedin_post	LangGraph StateGraph; 4 specialist tools (trend, geo, role, qa); MLflow autolog tracing

3.2 End-to-End Data Flow

The table below traces a single job posting from raw web scrape through to the final LinkedIn post generated by the multi-agent system.

#	From	To	Mechanism
1	Google / Amazon career pages	raw_openings (Delta)	HTTP scrape → BeautifulSoup parse → MERGE ON (job_id, org)
2	raw_openings	clean_openings (Delta)	Normalise titles, locations, seniority, domain, job family; quality gates
3	raw_openings	quarantine_openings (Delta)	Records failing quality checks; structured quarantine_reason field
4	clean_openings	job_sentence_chunks (Delta)	Sentence-window chunking; org-aware section parse; MERGE ON (doc_id, chunk_id)

5	job_sentence_chunks	job_chunks_index (VS)	DELTA_SYNC triggered sync; databricks-gte-large-en embeds sentence column
6	User NL query	Retrieved chunks	Hybrid ANN+BM25 → VS pre-filter → MMR re-rank → format_context()
7	Retrieved chunks	Tool outputs (ToolMessage)	LangGraph orchestrator routes to trend / geo / role / qa agents
8	Tool outputs	final_answer + linkedin_post	Orchestrator LLM synthesises all tool results into two deliverables

4. Component Design

4.1 Ingestion Layer

Two scrapers collect job listings in parallel. Both register with WatermarkManager on startup and query raw_openings before scraping to prevent re-processing the same file_dt.

Google Careers — scrape_google_careers.py

- Iterates up to 10 pages per search term: data, ai, agentic, software.
- Extracts title, organisation, location, position type, about, qualifications and responsibilities via BeautifulSoup CSS selectors.
- MERGE ON (job_id, organization) — deduplicates into raw_openings.
- On skip / no data / failure: marks watermark accordingly; never advances last_processed_dt.

Amazon Careers — scrape_amazon_careers.py

- Queries amazon.jobs JSON search API across 7 trend terms with pagination (offset-based).
- Fetches full HTML job description per listing; rate-limited with exponential back-off on HTTP 429.
- Merges on (id_icims, organization); HTTP 4xx errors are swallowed gracefully per listing.

Watermark Manager — pipeline_watermark.py

Shared across all stages. Tracks (org_key, stage) pairs in pipeline_watermarks Delta table. Provides mark_stage_success, mark_stage_failed, mark_skipped, mark_no_data, and status. The pipeline_watermark_history table provides an append-only, never-overwritten audit trail.

4.2 Cleaning Layer — clean_loaded_data.py

Reads raw_openings within the per-org watermark window. Applies the following transformations:

- Title normalisation — canonical mapping of job title variants.
- Location parsing — city and country extracted from free-text job_location.

- Seniority detection — regex + heuristics to classify Engineer I / II / III / Senior / Staff.
- Job family classification — Data Engineering, Software Engineering, Data Science, AI/GenAI, Data Center Operations.
- Domain / team extraction — Fintech, AWS, Advertising, etc. for Amazon roles.
- Intern flag — boolean is_intern derived from title and description keywords.
- Quality gates — records missing title or description are quarantined with a structured quarantine_reason.

4.3 Chunking Layer — chunker.py

Sentence-window strategy

Each chunk stores two artefacts: the focal sentence (embedded into the vector index) and the window text (passed to the LLM for generation context). This gives dense, precise retrieval while keeping rich context for generation.

Chunking Parameters

- window_size = 2: focal sentence S[i] stored with S[i-2] through S[i+2] as window_text.
- sentence = S[i] — the text embedded into the vector index (clean, dense).
- window_text = S[i-2] ... >>>S[i]<<< ... S[i+2] — passed to the LLM for context.
- Section detection: Google and Amazon JDs split by section headers before chunking.
- Boilerplate removal: Amazon equal-opportunity and pay-range paragraphs stripped prior to chunking.
- MERGE ON (doc_id, chunk_id) — idempotent; re-running chunker never produces duplicate rows.
- min_sentence_len = 20 chars — fragments shorter than this are discarded.

4.4 Embedding & Vector Search

Databricks Vector Search (DELTA_SYNC) automatically re-embeds job_sentence_chunks whenever a sync is triggered. No manual embedding loop is required — the sync engine calls the embedding model on the source Delta table.

- Index type: DELTA_SYNC with TRIGGERED pipeline — syncs are explicit, not continuous.
- Embedding model: databricks-gte-large-en — Databricks-hosted; no external API key required.
- Embedded column: sentence (focal sentence only for clean, dense embeddings).
- Metadata synced: org_key, job_family, domain, seniority, is_intern, country, trend_dt, file_dt — all available as filter columns at query time.
- Sync polling: vs_sync.py polls detailed_state every 30 seconds (up to 60 minutes). Watermark advances only after ONLINE_NO_PENDING_UPDATE is confirmed.

4.5 Retrieval Layer — retriever.py

Three-stage retrieval pipeline

Stage 1 — VS native pre-filter: org_key, job_family (normalised), country, section_type, trend_dt range, is_intern applied inside the vector search query for maximum performance.

Stage 2 — Hybrid candidate pool: retrieves top_k × 4 candidates (default 60) using HYBRID query (dense ANN + BM25 lexical). Improves recall for rare technical terms such as dbt, Iceberg, and Medallion that dense-only search misses.

Stage 3 — MMR re-ranking: Maximal Marginal Relevance selects from the candidate pool to balance relevance and diversity. Lambda = 0.5. Prevents all returned chunks from originating from the same near-duplicate job posting.

4.6 Multi-Agent Layer — build_agents.py

A LangGraph StateGraph orchestrates four specialist tools. The orchestrator LLM decides which tools to invoke based on the user's natural language query — it may call one, several, or all four in sequence.

Agent	Tool name	Responsibility
Orchestrator	`LangGraph graph`	Routes query to specialist tools; synthesises final_answer and linkedin_post from all ToolMessage outputs.
Trend agent	`trend_analysis`	Analyses skill frequency, technology mentions, Python/AWS/SQL demand across retrieved chunks.
Geo agent	`geo_analysis`	Identifies hiring locations, country distribution, regional skill differences (e.g. US vs India).
Role agent	`role_analysis`	Compares seniority levels (I / II / III / Senior / Staff), responsibilities, and domain teams.
QA agent	`qa_tool`	Open-ended factual Q&A — answers specific questions not covered by the three specialist agents.

5. Delta Table Schema Summary

Table	Writer	Type	Purpose
`raw_openings`	Scrapers	Delta + CDF	Source of truth; one row per scraped job posting
`clean_openings`	clean_loaded_data.py	Delta + CDF	Normalised; all orgs merged; feeds chunker
`quarantine_openings`	clean_loaded_data.py	Delta (no CDF)	Bad records with structured quarantine_reason
`job_sentence_chunks`	chunker.py	Delta + CDF	One row per sentence chunk; MERGE idempotent
`pipeline_watermarks`	WatermarkManager	Delta + CDF	One row per (org_key, stage); UPSERT on every run

pipeline_watermark_history	WatermarkManager	Delta (append)	Append-only audit log of every run attempt
job_chunks_index	embedding.py	Databricks VS	DELTA_SYNC index; sentence column embedded
agent_query_results	08_run_queries.py	Delta (append)	Saved query results for reporting and dashboards

6. Watermark & Idempotency Design

The watermark system is the backbone of the pipeline's safety guarantees. Every stage reads a lower-bound watermark (last successfully processed date) and an upper-bound watermark (the minimum of all upstream stages' watermarks) before deciding what data to process.

Stage	org_key	Gate	Advance condition
normalize	google / amazon	lower = last normalize wm	Only on successful MERGE to clean_openings
chunk	_pipeline	upper = MIN(all normalize wm)	Only after all chunks written; idempotent MERGE
vectorize	_pipeline	upper = chunk wm	Only after ONLINE_NO_PENDING_UPDATE confirmed

Watermark Safety Rules

- Failed runs leave the watermark unchanged — the next run automatically retries the same window.
- Adding a new org (e.g. Microsoft) tightens the chunk upper bound until its normalize watermark catches up.
- Scraping stages never advance last_processed_dt — that responsibility belongs to clean_loaded_data.py.
- The history table is append-only and never overwritten — provides complete lineage for every run attempt.
- mark_stage_success advances the watermark only after the MERGE or sync has fully committed.

7. Technology Stack

Category	Technology	Version / config	Role in pipeline

Compute	Databricks (serverless)	Job cluster	All pipeline stages; auto-terminates post-run
Storage	Delta Lake	CDF enabled	ACID, time-travel, change data feed on all tables
Embedding model	databricks-gte-large-en	Databricks-hosted	Sentence-level embeddings; no external API key needed
Vector search	Databricks VS	DELTA_SYNC / TRIGGERED	Auto-syncs from Delta; hybrid ANN+BM25 at query time
Agent framework	LangGraph	StateGraph	Multi-agent orchestration with typed message state
LLM client	databricks-langchain	ChatDatabricks	Unified LLM interface; MLflow autolog tracing
Web scraping	requests + BeautifulSoup	Python 3	Google (HTML parse) and Amazon (JSON API) scrapers
Watermark store	Delta + WatermarkManager	Custom Python	Per-stage idempotency; append-only history audit log
MLOps / Tracing	MLflow	langchain.autolog()	Token counts, latency, tool call traces per run

8. Running the Pipeline

8.1 One-time bootstrap

- Run bootstrap_watermark.py to create all Delta tables and seed initial watermark rows.
- Ensure the Databricks VS endpoint (zachy_vs) is available — vs_index.py creates it idempotently.
- Install packages: %pip install --upgrade databricks-langchain langchain langgraph

8.2 Daily pipeline execution order

1. scrape_google_careers.py (run in parallel with step 2)
2. scrape_amazon_careers.py
3. clean_loaded_data.py
4. chunker.py
5. embedding.py (triggers VS sync; waits for ONLINE_NO_PENDING_UPDATE)
6. 08_run_queries.py (run queries; results saved to agent_query_results Delta table)

8.3 Example query

Basic usage — call run_query() with a natural language question:

```
from agents import run_query
result = run_query("What are the top Python and Spark skills Amazon is hiring for?")
print(result["final_answer"])
print(result["linkedin_post"])
```

9. Sample Pipeline Output

The following is the LinkedIn post generated by the multi-agent system from the March 29, 2026 pipeline run, based on analysis of job postings scraped from Amazon and Google careers:

Generated LinkedIn Post

14 job postings require Python expertise in just one month, highlighting the consistent demand for data engineers with strong programming skills.

AWS services are also in high demand, with 12 mentions, surpassing other skills like data modelling and ETL pipelines.

Top skills include Python, AWS services, data modelling, ETL pipelines, and SQL (9 occurrences). The dominance of the AWS stack is evident — 12 mentions vs 0 for GCP.

Experience requirements vary: 4 postings require 1+ year; 7 require 3+ years.

What will be the impact of emerging technologies like GenAI and Agentic AI on the job market for data engineers?

#DataEngineering #AWS #Python #JobMarket #DataModeling #ETLPipelines #SQL

10. Acknowledgements

This pipeline was developed as part of the DataExpert bootcamp programme. Special thanks to Umar (@eumarassis_29883) and the entire DataExpert team for their expert guidance, high-quality course content, and community support throughout the project.

End of System Design Document · Job Market Intelligence Pipeline · March 2026